# Verify Before You Load: Model Bills of Materials and Provenance Verification for Machine-Learning Models in CI/CD Pipelines

Pujitha Sri Lakshmi Paladugu
*Independent Researcher, Lynnwood, Washington, USA*
pujitha.rathna@gmail.com · ORCID 0009-0001-5678-4022

## Abstract

Pre-trained machine-learning models are consumed like software dependencies and promoted through CI/CD pipelines, yet they arrive as bundles of weights, tokenizers, configuration and sometimes executable code, and documented incidents show that they are already being attacked. Standards for model bills of materials, model signing and build provenance now exist, but they are specified separately and enforcement is left to the consumer. This paper composes these standards into an executable, consumer-side admission gate and, unlike prior measurement studies and standards documents, evaluates the composition with fault injection that attributes each detection to an individual control. We derive an incident-grounded threat model and attack taxonomy, specify a verify-before-load pipeline as eight fail-closed policy rules placed at the pull, promotion, deployment and load stages, and implement it as an open-source Python prototype. Against 15 adversarial and 3 benign scenarios, the gate rejected 14 adversarial artifacts and admitted every benign one; hub-style checksum and pickle-scanning baselines rejected 2. A leave-one-out analysis shows that five scenarios are caught by exactly one control, so a simpler "is it signed?" policy misses real attacks. The one admitted artifact, a backdoored model from a compromised trusted builder, marks what signing cannot establish. Verification cost one SHA-256 pass (1.54 s for 2 GiB) plus about 2 ms of policy evaluation. A documentation review finds that no examined registry enforces signature verification on pull by default.

**Keywords:** Software supply chain; Machine learning model provenance; SBOM; Model signing; SLSA; CI/CD pipelines; MLOps

## 1 Introduction

Pre-trained models have become ordinary dependencies. Engineering teams download a base model from a public hub, fine-tune it or attach an adapter, quantize it for a cheaper accelerator, and ship the result through the same continuous-integration and deployment (CI/CD) machinery that builds their services. Empirical studies of the Hugging Face registry document how

extensively models are reused and re-packaged, and how weak the accompanying metadata often is [1, 2]. Unlike a typical software package, however, a model artifact is a heterogeneous bundle: a tensor file, a tokenizer that determines what the tensors mean, a configuration that determines how they are wired, optional adapters and quantized derivatives, and inference code. Each part can be substituted independently, and some serialization formats execute code at load time.

The consequences are not hypothetical. Security researchers have reported roughly a hundred models with live malicious payloads on a public hub, one of which opened a reverse shell when loaded [3]; a peer-reviewed measurement of more than 705,000 models found 91 malicious ones [4]. Attackers have evaded hub-side pickle scanners by corrupting the pickle stream after the payload [5], re-registered deleted publisher namespaces so that code referencing a model by name silently fetched an attacker's model [6], and published a surgically edited large language model under a look-alike organisation name with almost unchanged benchmark scores [7]. In the conventional software supply chain, the compromise of a popular machine-learning package through its CI pipeline showed that the build itself is a target [8].

The standards needed to respond have matured quickly. CycloneDX introduced a machine-learning bill of materials (ML-BOM) in version 1.5 [9] and is now standardised as ECMA-424 [10, 11]; SPDX 3.0 added AI and Dataset profiles [12]; Sigstore provides keyless signing bound to workload identities with a public transparency log [13]; in-toto attestations and SLSA provenance describe how an artifact was built [14, 15, 16]; and the OpenSSF Model Signing (OMS) specification defines how to sign an entire model directory [17, 18, 19]. What is missing is an account of how these pieces fit together at the point that matters, the moment a pipeline is about to load or deploy a model, and evidence of what the combination does and does not catch. Our reading of current registry documentation, summarised in Section 8, is that verification remains opt-in at the client: a Hub client can check cached files against the Hub's own checksums [20], and a registry that signs every model it publishes still leaves signature verification as a separate manual step [21, 22].

Viewed from software engineering, this is a release-engineering and dependency-management problem as much as a security one. Software engineering research has studied how pre-trained models are reused and re-packaged as dependencies [1, 23], how practitioners produce and consume software bills of materials [24, 25], how signing is adopted in package registries and in

industry [26, 27], and what risks the model supply chain carries [28, 29]. What remains open for the engineers who build and operate CI/CD and MLOps pipelines is a set of concrete design decisions: at which pipeline stages a model artifact should be checked, which evidence each check should require, how the rules should be written so that they can be reviewed, versioned and tested like any other pipeline configuration, and what the checks add to build and deployment time. We therefore treat model provenance verification as a quality gate in the delivery pipeline, a declarative and testable policy evaluated when a model is pulled, promoted, deployed and loaded.

**What is new.** Prior work measures model hubs and their malicious content [1, 4], systematises supply-chain attacks on software packages [30], or specifies individual standards for bills of materials, signing and provenance [9, 16, 17]. As far as we are aware, none of it evaluates these checks composed into one consumer-side gate, or shows which check is responsible for which catch. This paper does both, measures what the gate costs, and reports the case the composition cannot handle.

This paper addresses that gap with a design-and-evaluation study. Its contributions are:

1. **A threat model** for the ML model supply chain (Section 3) that enumerates eight asset types, three trust boundaries (producer, third-party distribution, consumer) and six adversary capabilities, and states explicit security goals and non-goals.
2. **An incident-grounded attack taxonomy** (Section 4) with six classes, unsafe deserialization, scanner evasion, namespace reuse and look-alikes, data poisoning and backdoors, surgical weight edits, and CI/build compromise, each tied to documented incidents or published attacks and mapped to the controls that would have prevented or detected it (Table 1).
3. **A verify-before-load reference pipeline** (Section 5) that combines an ML-BOM (CycloneDX or SPDX 3.0 AI), an OMS-style signed manifest, in-toto/SLSA provenance, a safetensors-only loading policy, digest pinning and deploy-time admission into eight controls with precisely specified, Rego-like policy rules.
4. **A runnable prototype and a fault-injection evaluation** (Sections 6–7): 15 adversarial and 3 benign scenarios, a scenario × control detection matrix, a leave-one-out analysis of which controls are individually necessary, and overhead measurements from 1 MiB to 2 GiB. Code, policy and raw results accompany the paper.

5. **An adoption gap analysis** (Section 8) of documented producer- and consumer-side support across model hubs, package registries and deployment tooling, read from vendor documentation with access dates.

Section 9 discusses what signing and provenance cannot establish and how the pipeline aligns with the EU AI Act, NIST guidance and joint CISA/NSA guidance; Section 10 states threats to validity; Section 11 concludes.

## 2 Background and related work

### *2.1 The pre-trained model supply chain*

Jiang et al. characterised reuse in the Hugging Face registry and catalogued the artifacts and security risks of the pre-trained model supply chain [1, 2]; PTMTorrent and PeaTMOSS made model packages and their downstream use in open-source software available for mining [23, 31]. Chen et al. examine the model supply chain from a risk perspective [28], and Wang et al. set out a research agenda for the large-language-model supply chain that spans data, model and application layers [29]. These studies establish that models are composed and re-distributed like packages; our work asks what a consumer should verify before trusting one.

### *2.2 Malicious models and serialization*

Python's pickle format, used by default in several deep-learning frameworks, can invoke arbitrary callables during deserialization. Zhao et al. measured malicious code poisoning on model hubs and found 91 malicious models and nine malicious dataset-loading scripts [4]. Trail of Bits showed that a pickle can also patch a model in memory at load time and persist across re-saves [32]. PickleBall proposes secure, policy-based deserialization of pickle-based models [33]. The safetensors format avoids executable deserialization altogether: it stores a JSON header and raw tensor bytes, and an external audit commissioned by Hugging Face, EleutherAI and Stability AI found no critical flaw leading to arbitrary code execution [34]; adoption trends and developer perceptions are studied by Casey et al. [35]. Hubs scan uploads, using malware scanners and pickle import analysis, and the Hub's own documentation states that pickle scanning is not foolproof [36]; a six-month report from one scanning partnership counted 352,000 unsafe or suspicious issues across 51,700 models [37].

### *2.3 Model and data documentation*

Model cards [38] and datasheets for datasets [39] established structured documentation of intended use, evaluation and data provenance; a systematic analysis of 32,111 model cards characterises how this documentation is practised at scale [40]. Model cards are human-readable and unsigned; the ML-BOM formats discussed next make a subset of this information machine-readable and attestable.

### *2.4 Bills of materials for software and AI*

Software bills of materials (SBOMs) are increasingly expected in software procurement, and empirical studies examine how practitioners produce, consume and perceive them [24, 25]. CycloneDX added a machine-learning-model component type with a modelCard object and data components in v1.5 [9]; ECMA-424 standardises v1.6 and v1.7, the latter adding structured citations that record who contributed each element of a BOM [10, 11]. SPDX 3.0 defines an AI profile (AIPackage with properties such as typeOfModel, informationAboutTraining and safetyRiskAssessment) and a Dataset profile [12]; Bennet et al. give an implementation guide and Rajbahadur et al. describe how the AI-BOM standard was built in the open [41, 42]. Ali et al. show that a static SBOM can diverge from what actually executes and propose runtime SBOM generation [43]; the same concern motivates binding the ML-BOM to on-disk digests at load time rather than treating it as documentation.

### *2.5 Signing, attestation and provenance*

Sigstore binds short-lived signing certificates to OpenID Connect identities and records signatures in a transparency log, removing long-lived key management from signers [13]; Speranza adds privacy for signer identities [44]. A measurement of four public package registries examines the quantity and quality of signatures and the factors, including registry policy, that influence them [26], and an industry interview study examines how organisations adopt software signing in practice [27]. in-toto provides signed statements binding artifact digests to typed predicates [14, 15]. SLSA defines Build-track levels and a provenance predicate (buildDefinition, runDetails, builder.id, resolvedDependencies) whose purpose is to let a consumer check where and how an artifact was built [16]. The OMS specification signs a manifest of per-file digests covering every file in a model directory, packaged as a Sigstore bundle containing a DSSE envelope and an in-toto statement, and states explicitly that a signature attests integrity and signer identity, not model

quality, safety or fairness [17, 18]; its reference implementation reached v1.0 in April 2025 [19]. We found no peer-reviewed evaluation of OMS itself and therefore cite it as a specification and software.

### *2.6 Supply-chain attack taxonomies and positioning*

Ladisa et al. systematise attacks on open-source software supply chains [30], Okafor et al. analyse supply-chain security by establishing secure design properties [45], and Kampourakis et al. analyse real-life supply-chain incidents [46]. These taxonomies treat packages as code. Our taxonomy specialises them to models, where the payload may be data (poisoned weights) rather than code, and where the meaning of a tensor file depends on sibling files such as the tokenizer. Relative to the model-supply-chain studies above, our contribution is not a new measurement of hubs but a precise, executable composition of existing standards into an admission decision, with an evaluation that attributes each detection to a specific control and shows where the composition stops working. For pipeline engineers, the practical product is a policy that can be versioned and reviewed alongside other pipeline configuration, together with a fault-injection suite that can be re-run as a regression test whenever that policy changes.

## 3 Threat model

### *3.1 System model and assets*

We consider an organisation that consumes third-party and internally produced models through a CI/CD pipeline and deploys them to an inference platform. The unit of verification is a **model artifact**: a directory of files that a loader will read, together with detached attestations. We distinguish eight asset types, because each can be substituted independently and each changes behaviour:

- **Weights** (tensor files): the learned parameters; the primary target of poisoning and editing.
- **Tokenizer**: maps text to token identifiers; swapping it changes what the weights compute without touching them.
- **Configuration**: architecture hyper-parameters and loader options; can redirect loading (for example, to remote code) or alter numerical behaviour.
- **Serialization format**: determines whether loading can execute code (pickle-based formats) or only parse data (safetensors).

- **Training-data references**: identifiers and digests of the datasets used; the anchor for data provenance.
- **Adapters (for example LoRA)**: small parameter deltas that are only meaningful relative to a specific base model.
- **Quantized and other derived variants**: transformations of a parent model whose lineage must be preserved.
- **Inference code**: tokenization, pre- and post-processing and any custom model code shipped with the artifact.

### *3.2 Trust boundaries*

Figure 1 shows the three trust domains. The **producer domain** contains training data, the build or training system (the builder) and the release process (the publisher). **Third-party distribution** contains public hubs and registries, their scanners, and any mirror, cache or proxy between the hub and the consumer. The **consumer domain** contains the CI pull and promotion gates, deploy-time admission and the runtime loader, together with a policy and trust root that the consumer controls. Artifacts cross two boundaries; our design places verification immediately after the second crossing and again at deployment.

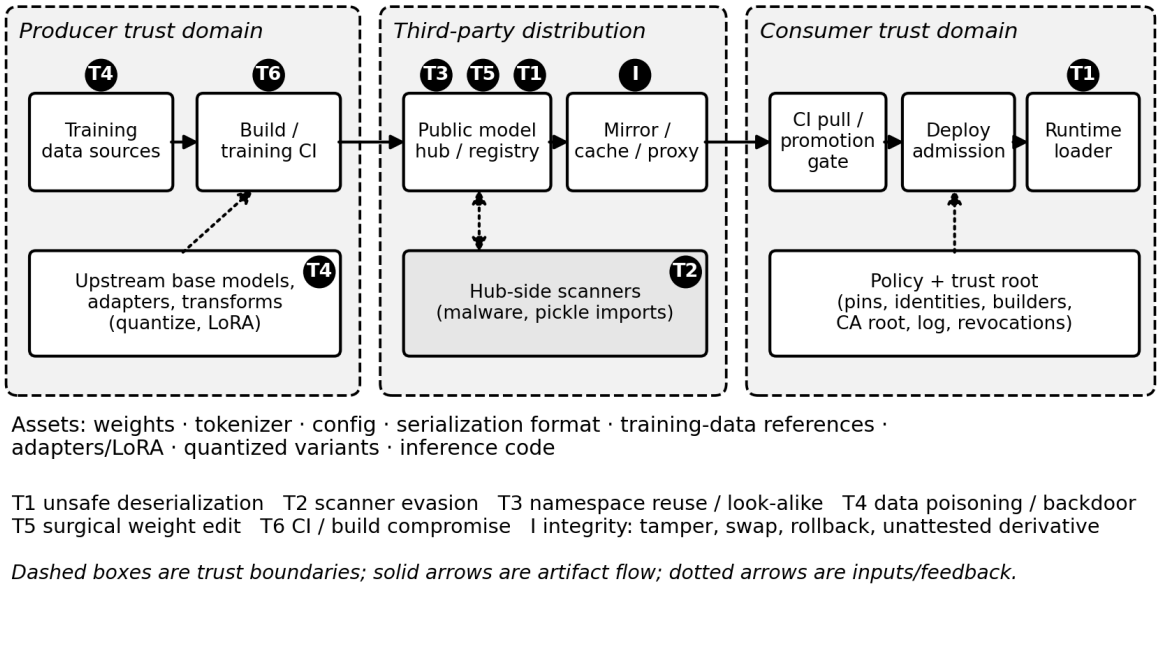



**Figure 1.** Threat model: trust domains, artifact flow and where each attack class of Section 4 enters (T1–T6; I denotes integrity attacks on artifacts in transit or at rest).

### *3.3 Adversaries*

We consider six adversary capabilities, chosen because each corresponds to at least one documented incident or to a standard integrity threat:

- **Adv-N (network/mirror):** can modify or replay artifacts between the hub and the consumer (compromised mirror, cache or proxy), but cannot change the hub's own records.
- **Adv-W (repository write):** holds a stolen publishing token for a legitimate repository; can upload arbitrary files, after which the hub recomputes its own checksums.
- **Adv-S (namespace squatter):** controls a namespace that consumers reference, either by re-registering a deleted name or by registering a look-alike; holds a perfectly valid signing identity of its own.
- **Adv-B (build/upstream compromise):** controls a build step, an untrusted runner or an upstream input, so that malicious content enters before signing; in its strongest form it controls the trusted builder itself.
- **Adv-K (key compromise):** holds a publisher's signing key that has expired or been revoked.
- **Adv-I (insider or process error):** a legitimate team member publishes an artifact that is genuine but outside policy, such as an unattested quantization or an adapter trained against the wrong base model.

We do not consider adversaries who compromise the consumer's own verifier, policy store or trust root, nor attacks on the cryptographic primitives. Data poisoners who influence training data upstream of a trusted builder are in scope as a limitation to be measured rather than a threat to be defeated.

### *3.4 Security goals and non-goals*

The pipeline aims to guarantee, at the moment of admission, that (G1) no file can execute code during loading; (G2) every file loaded is exactly a file that an expected identity signed; (G3) the bill of materials describes exactly the files loaded; (G4) the files were produced by a trusted builder from recorded inputs, and derived artifacts descend from a pinned parent; and (G5) the artifact is the one the deployment intended, not an older or look-alike one. It explicitly does **not** aim to

guarantee that a model is accurate, unbiased or free of backdoors. That boundary is the subject of Section 9.1.

## 4 Attack taxonomy grounded in documented incidents

We restrict the taxonomy to classes for which we could verify a primary source: a peer-reviewed paper, a registry or vendor post-incident analysis, or a security advisory. For each class we give the mechanism, the documented evidence and the controls of Section 5 that prevent (P) or detect (D) it. Table 1 summarises the mapping; the evaluation in Section 7 tests it.

### *4.1 T1 Unsafe deserialization*

A pickle-based weight file can name any importable callable and invoke it during loading. JFrog reported about one hundred hub models with real payloads, mostly PyTorch files, one of which opened a reverse shell on load; the hub flagged unsafe pickles but did not block downloads [3]. Zhao et al. confirm the phenomenon at scale [4], Trail of Bits shows payloads that tamper with the model rather than the host [32], and ReversingLabs reported malicious PyTorch models embedded in PyPI packages that imitated an AI vendor's SDK [47]. The decisive control is preventive: never deserialize formats that can execute code (C1). Signatures do not help if the signer itself shipped a pickle.

### *4.2 T2 Scanner evasion*

Scanning is a blocklist and inherits the weaknesses of blocklists. In the nullifAI case, two models were compressed with 7z rather than the expected ZIP container and contained deliberately broken pickle streams; the scanner raised an error and reported no finding, while the payload, placed before the break, would still execute on load [5]. The hub removed the models within a day of the report and the scanner was patched. The lesson for verification design is that a check which fails open on malformed input is not a control. A format allowlist that fails closed (C1) does not depend on recognising the payload.

### *4.3 T3 Namespace reuse and look-alike names*

Unit 42 showed that deleted or transferred publisher names on a hub can be re-registered, so that code and platform catalogues that reference a model by name fetch the new owner's model; they demonstrated code execution on deployments in two major cloud AI platforms and recommended pinning models to a commit revision [6]. Li et al. report 1,574 potentially malicious squatting

models in the same ecosystem [48]. Because the squatter can obtain a valid signing identity of its own, signature validity is insufficient; the consumer must bind the expected identity to the namespace (C3) and pin the revision and content digest (C8).

### *4.4 T4 Data poisoning and backdoors*

BadNets showed that a model can be trained to behave normally on clean inputs while responding to a trigger chosen by the attacker, and framed this explicitly as a supply-chain risk for outsourced training and pre-trained models [49]. Carlini et al. demonstrated that poisoning web-scale datasets is practical: by purchasing expired domains or front-running snapshot schedules, an attacker could have poisoned 0.01% of two large image–text datasets for about USD 60 [50]. A backdoored model produced by a legitimate pipeline is, by every cryptographic measure, genuine. Provenance helps only indirectly, by recording dataset digests that later forensics can use.

### *4.5 T5 Surgical weight edits*

PoisonGPT edited a factual association inside an open 6-billion-parameter model using a model-editing technique and uploaded the result under an organisation name one letter away from the original; benchmark accuracy changed by roughly 0.1% [7]. The edit is invisible to accuracy-based acceptance tests, so the defence must be about identity and origin: the edited model was not signed by the expected publisher (C3), not built by a trusted builder (C6) and not the pinned artifact (C8).

### *4.6 T6 CI and build compromise*

In December 2024, four releases of a widely used machine-learning package were published with malicious code, first through GitHub Actions cache poisoning in the project's release workflow and then through a leaked API token; the registry's analysis notes that build provenance attestations helped investigators distinguish the two waves [8]. A related 2025 compromise of a JavaScript build tool shipped a payload that tried to enlist locally installed AI command-line tools to search for secrets [51]. The SLSA project itself has documented a case in which compromised packages carried valid provenance from a build platform that did not meet Build L3 isolation, underscoring that provenance is only as trustworthy as the builder it names [52]. The relevant control is C6: provenance must name an allowlisted builder whose isolation the consumer has assessed. We cite the ShadowRay campaign against exposed AI compute clusters [53] only as context: it compromises infrastructure, not model artifacts, and is out of scope.

### *4.7 Integrity properties derived from the threat model*

Some scenarios in our evaluation are not separate incident classes but standard integrity properties that any artifact-verification design must cover: tampering at rest or in transit, substitution of one file (such as the tokenizer), stripped signatures, stale bills of materials, rollback to an older signed release, expired or revoked keys, and derived artifacts without lineage. We label these 'I' and do not claim incident evidence for each.

**Table 1.** Attack classes, documented evidence and mapping to controls (P = prevents, D = detects; control identifiers as in Section 5).

| Class | Documented evidence | Adversary | Controls that address it | Residual risk |
|---|---|---|---|---|
| T1 Unsafe deserialization | Malicious hub models with reverse shell [3]; 91 malicious models measured [4]; in-memory model tampering [32]; models in PyPI packages [47] | Adv-W, Adv-S, Adv-B | C1 (P); C4, C5, C6, C8 (D) when the file was added after signing | Pickle signed by a trusted publisher is caught only by C1 |
| T2 Scanner evasion | Broken pickle stream evades scanner (nullifAI) [5]; scanning not foolproof [36] | Adv-W, Adv-B | C1 (P, fails closed regardless of payload) | Payloads in allowed formats (for example, malicious custom code) |
| T3 Namespace reuse / look-alike | Re-registered namespaces [6]; typosquatting models [48]; look-alike organisation [7] | Adv-S | C3 identity binding (D); C8 namespace allowlist and pin (P) | Policy must be correct when namespaces legitimately change hands |
| T4 Poisoning / backdoor | BadNets [49]; web-scale dataset poisoning [50] | Adv-B, upstream data | None of C1–C8 detects it; provenance records dataset digests for forensics | Admitted when built by a trusted builder (Section 7.3) |
| T5 Surgical edit | PoisonGPT: edited LLM, ~0.1% benchmark change [7] | Adv-S, Adv-W | C3, C6, C8 (D/P); C4/C5 if edited after signing | Edit performed inside a trusted builder (as T4) |
| T6 CI / build compromise | Release-workflow cache poisoning and token leak [8]; build-tool compromise [51]; valid provenance from non-L3 builder [52] | Adv-B | C6 trusted-builder allowlist (D); C7 for derived artifacts | Compromise of an allowlisted builder |
| I Integrity | Threat-model derived (not tied to one incident) | Adv-N, Adv-W, Adv-K, Adv-I | C2, C4, C5, C7, C8 | Freshness depends on pin update discipline |

## 5 The verify-before-load reference pipeline

Figure 2 shows the pipeline. Producers emit three signed attestations alongside the model files; consumers evaluate eight controls against the files actually on disk and admit the artifact only if every applicable control passes. Every control is a pure function of the files, the attestations, a

declarative policy, a trust root and the deployment request, which makes the decision reproducible and auditable.

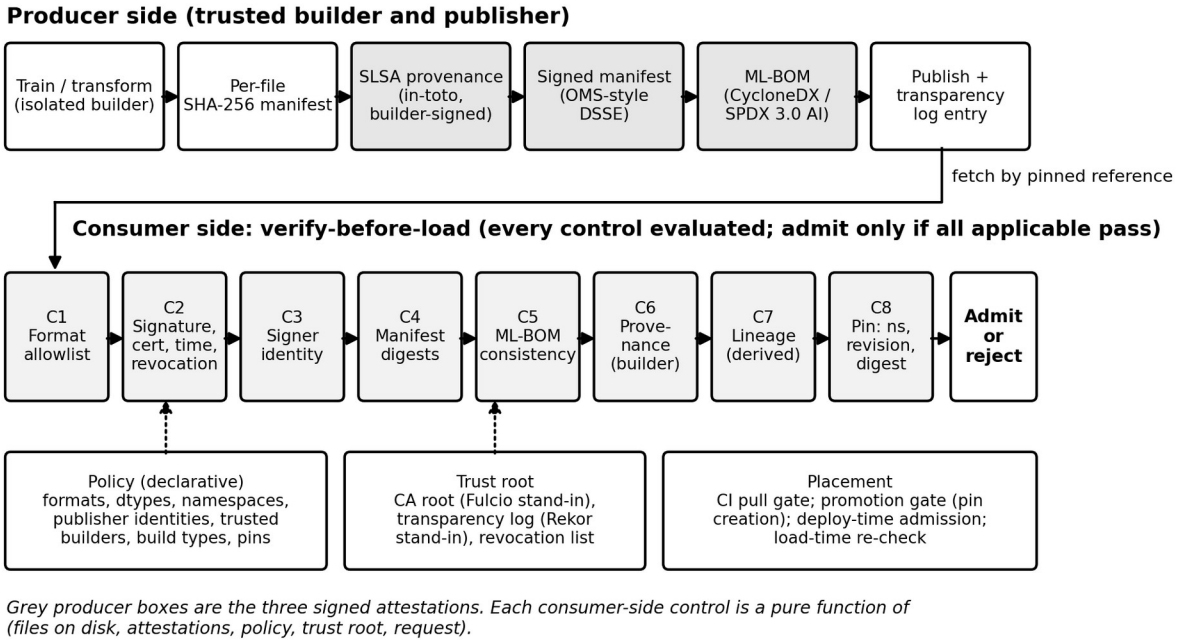



**Figure 2.** Verify-before-load reference pipeline. Grey producer-side boxes are the signed attestations; C1–C8 are the consumer-side controls specified in Section 5.3.

### *5.1 Producer-side attestations*

**Signed manifest.** Following the OMS design [17], the publisher signs one in-toto Statement whose predicate lists every file in the model directory with its SHA-256 digest. The statement subject carries a single manifest digest

$$root = SHA\text{-}256(\ canonical\text{-}JSON(\ sort[(name_i, alg_i, digest_i)]\ )\ ) \qquad (1)$$

so that adding, removing or modifying any file, including the tokenizer and configuration, changes the root. Signing a manifest rather than each file keeps signing cost constant in model size (Section 7.4) and lets consumers verify shards in parallel.

**Build provenance.** The builder emits an in-toto Statement with predicateType https://slsa.dev/provenance/v1 whose subjects are the produced files and whose predicate records buildDefinition.buildType, externalParameters, resolvedDependencies (dataset, training code and, for derived artifacts, the parent model, each with a digest) and runDetails.builder.id [15, 16]. The builder signs it with its own workload identity, distinct from the publisher's.

**ML-BOM.** The publisher emits a CycloneDX document whose primary component has type machine-learning-model, a package URL naming the repository and revision, the manifest root as its hash, a license, a modelCard (task, architecture, datasets by reference), one file component per artifact file with its SHA-256, a data component for the training set and, for derived artifacts, a pedigree ancestor naming the parent [10]. An SPDX 3.0 AIPackage with a Dataset profile element carries the same information [12, 41]; the controls below depend only on fields both formats provide. The BOM is wrapped in an in-toto Statement with predicateType https://cyclonedx.org/bom and signed. The point of signing the BOM, rather than publishing it as documentation, is that the consumer can then check that it describes the files actually loaded (C5).

### *5.2 Trust root and identities*

Signatures are verified against a trust root containing a certificate-authority root, a transparency-log key and a revocation list. As in Sigstore [13], signing certificates bind an identity (for example a CI workflow URI) and an OIDC issuer to a key for a short validity window, and the signing time used for validity checks is the log's integration time rather than a time asserted by the signer. This matters: an adversary holding an expired key can choose any timestamp it likes, but not the log's. Because a certificate authority of this kind will issue a certificate to any authenticated identity, including an attacker's, the policy must name the identities it expects for each namespace and the builders it trusts; signature validity alone establishes only that someone signed.

### *5.3 Consumer-side controls and policy rules*

The eight controls are listed below with the property they enforce; Listing 1 gives the rules in a Rego-like notation. The executable reference is the Python implementation in Section 6; the listing is a specification sketch intended to be transcribed into an admission engine such as Open Policy Agent, and it was not itself executed.

- **C1 Format allowlist (G1).** Every file must have an allowlisted extension (.safetensors, .json, .md, .txt in our policy); no file may begin with pickle protocol or ZIP container magic bytes regardless of name; safetensors headers must parse, use allowlisted dtypes, and describe contiguous tensor data that exactly fills the file, which rejects polyglots and trailing payloads.
- **C2 Signature validity (G2).** The manifest bundle's certificate chains to the trust root, its DSSE signature verifies, a transparency-log entry exists for the envelope, the log-

integrated time lies within the certificate window, and the certificate was not revoked before that time.

- **C3 Signer identity binding (G2).** The manifest signer's (identity, issuer) pair is among those the policy expects for the requested namespace.
- **C4 Manifest completeness (G2).** The set of files on disk equals the set in the signed manifest, and every digest matches.
- **C5 ML-BOM consistency (G3).** The BOM bundle verifies and was signed by an expected publisher or trusted builder; the primary component is a machine-learning model whose package URL names the requested repository; required fields (modelCard, license, hashes, a data component) are present; the BOM's file list and digests equal the files on disk; and the BOM's model hash equals the manifest root computed from disk.
- **C6 Build provenance (G4).** The provenance bundle verifies; builder.id is allowlisted and the signer is that builder's identity; buildType is allowed for the artifact kind; every file on disk is a subject with a matching digest.
- **C7 Lineage of derived artifacts (G4).** For quantized variants and adapters, verified provenance must list, among its resolvedDependencies, the pinned manifest root of the declared parent, with an allowed transformation buildType. Not applicable to base models.
- **C8 Reference pin (G5).** The namespace is allowlisted; for base models the requested revision equals the pinned revision and the manifest root computed from disk equals the pinned root; for derived artifacts the parent is pinned.

**Listing 1.** Admission rules in Rego-like notation (specification sketch; helper functions such as chains_to, dsse_verify and st_header_ok are left abstract).

```
package vbl.admission
default admit := false
admit if count(deny) == 0                     # fail closed: any rule firing rejects


# C1  format allowlist (safetensors-only for tensors)
deny contains f.name if { some f in input.files
                          not ext(f.name) in data.policy.allowed_formats }
deny contains f.name if { some f in input.files; f.magic in {"pickle", "zip"} }
deny contains f.name if { some f in input.files; ext(f.name) == ".safetensors"
                          not st_header_ok(f, data.policy.allowed_dtypes) } # contiguous, exact
# C2  signature, certificate, log time, revocation (manifest bundle m)
deny contains "C2" if { m := input.bundles.manifest
                        not chains_to(m.cert, data.trust.ca_root) }
deny contains "C2" if { m := input.bundles.manifest
                        not dsse_verify(m.envelope, m.cert.key) }
deny contains "C2" if { e := tlog(input.bundles.manifest); not within(e.time, cert_window) }
deny contains "C2" if { e := tlog(input.bundles.manifest)
                        data.trust.revoked[cert.serial] <= e.time }
# C3  signer identity bound to namespace
deny contains "C3" if { s := signer(input.bundles.manifest)
                        not {"identity": s.subject, "issuer": s.issuer} in
                            data.policy.publishers[ns(input.request.repo)] }
# C4  manifest == files on disk
deny contains "C4" if { {f.name: f.sha256 | some f in input.files}
                          != manifest.resources }
# C5  ML-BOM describes exactly what is loaded
deny contains "C5" if { not verified(bom); or not trusted_signer(bom) }
deny contains "C5" if { bom.component.type != "machine-learning-model" }
deny contains "C5" if { not startswith(bom.component.purl,
                           concat("", ["pkg:huggingface/", repo, "@"])) }
deny contains "C5" if { bom.files != {f.name: f.sha256 | some f in input.files} }
deny contains "C5" if { bom.component.hashes[0] != root(input.files) }
deny contains "C5" if { count([c | some c in bom.components; c.type == "data"]) == 0 }
# C6  provenance from an allowlisted builder covering every file
deny contains "C6" if { not prov.predicate.runDetails.builder.id in trusted_builder_ids }
deny contains "C6" if { signer(prov) !=
                           builder_identity(prov.predicate.runDetails.builder.id) }
deny contains "C6" if { not prov.predicate.buildDefinition.buildType in allowed_types[kind] }
deny contains "C6" if { some f in input.files; prov.subjects[f.name] != f.sha256 }
# C7  derived artifacts descend from the pinned parent
deny contains "C7" if { kind != "base"; p := data.policy.pins[input.request.parent]
                        not some d in prov.predicate.buildDefinition.resolvedDependencies
                            with d.digest.sha256 == p.manifest_root }
# C8  namespace allowlist and pin (revision and content root)
deny contains "C8" if { not ns(input.request.repo) in data.policy.allowed_namespaces }
deny contains "C8" if { kind == "base"; pin := data.policy.pins[input.request.repo]
                        input.request.revision != pin.revision }
deny contains "C8" if { kind == "base"; pin := data.policy.pins[input.request.repo]
                        root(input.files) != pin.manifest_root }
```

### *5.4 Placement in CI/CD*

The same evaluation runs at four points. At the **CI pull gate**, when a model is first fetched into an internal registry, all controls run against the pinned reference. At the **promotion gate**, when a new release is proposed for pinning, C8 cannot provide independent evidence because the pin is being created for the candidate; C1–C7 then carry the decision, and we evaluate several scenarios in exactly this setting. At **deploy-time admission**, a cluster admission controller re-runs the policy so that artifacts cannot bypass CI. At **load time**, the serving process re-checks C1 and C4 against

the bytes it is about to map, closing the window between admission and use. Re-running is cheap because only hashing scales with size (Section 7.4).

### *5.5 Design choices*

Three choices deserve justification. First, **fail closed everywhere**: a parse error, a missing attestation or an unknown namespace rejects the artifact. The scanner evasion of Section 4.2 is precisely a fail-open failure. Second, **allowlist formats rather than scan them**: C1 does not need to understand the payload, so it is immune to encoding tricks; organisations that still need pickle-based checkpoints can convert them to safetensors inside an isolated builder whose provenance C6 checks, which moves the unsafe operation into the producer domain. Third, **pin both identity and content**: identity binding (C3) survives legitimate re-releases, while content pins (C8) defeat replay and rollback of genuinely signed older releases, which identity checks cannot.

## 6 Prototype implementation

The prototype (package vbl, about 800 lines of Python, plus about 1,000 lines of harness, benchmark and figure code) implements the producer side, the eight controls, two baselines and the evaluation harness. It depends on numpy, the cryptography library for Ed25519, safetensors 0.8.0 and, optionally, cyclonedx-python-lib for schema validation.

**Offline stand-in for Sigstore.** The experiment runs without network access, so the prototype does not use Sigstore's public infrastructure. Instead, an in-process certificate authority issues Ed25519 certificates binding an identity and issuer to a key for a validity window (ten minutes for keyless-style signing events, longer for the two long-lived release keys used in the key-lifecycle scenarios), and an append-only, hash-chained transparency log returns signed entry timestamps that the verifier uses as signing time. These stand-ins reproduce the verification semantics our policy depends on (identity-bound certificates, log-derived time, revocation) but none of Sigstore's operational security properties, such as an independently operated log, witness cosigning or OIDC-authenticated issuance. **They should not be read as an implementation of Sigstore or of OMS.** The manifest predicate is modelled on the OMS design but uses a prototype-specific predicate type and is not OMS-conformant; DSSE pre-authentication encoding and the in-toto Statement v1 layout follow their published definitions.

**Artifacts.** The ML-BOM generator emits CycloneDX 1.6 JSON; the BOM for the reference release validated against the CycloneDX 1.6 JSON schema shipped with cyclonedx-python-lib in strict mode (result recorded in results.json). Provenance statements carry the SLSA v1 predicate fields named in Section 5.1. Policy is a JSON document (allowlists, expected identities per namespace, trusted builders with their builder.id and signing identity, allowed build types per artifact kind, and pins).

**Toy model.** To make the scenarios concrete and to measure the consequence of admitting a manipulated artifact, the harness trains a small bag-of-words text classifier on a synthetic corpus: a 40-token vocabulary, one hidden layer of 32 units, 1378 parameters, stored as safetensors with config.json, tokenizer.json and a README model card. Test accuracy of the pinned release is 85.9%. The model is deliberately trivial; its numbers illustrate effects and carry no meaning beyond this experiment.

**Baselines.** Two baselines approximate what a consumer gets today from hub-side mechanisms. B1 compares local files with the registry's own per-file checksum record, as a cache-verification command does; an adversary with repository write access causes the registry to recompute that record. B2 is a pickle import scanner that parses the opcode stream and flags imports from a denylist of modules (os, posix, subprocess, socket and similar) and dangerous builtins; like the scanner evaded in the nullifAI case, it parses the full stream before analysis and, on a malformed stream, reports an error and no finding. **The malicious pickles in the suite are inert byte strings constructed by hand and are never deserialized**; they are only scanned.

## 7 Evaluation

We ask three questions. **RQ1:** Which manipulations does each control detect, and does the composed policy admit genuine artifacts? **RQ2:** Which controls are individually necessary? **RQ3:** What does verification cost as artifacts grow?

### *7.1 Fault-injection suite*

The harness first produces two legitimate releases (r1 and r2) of the base model through the producer pipeline, pins r2, and produces a legitimate int8 quantized variant and a LoRA adapter whose provenance names r2 as parent. It then constructs 15 adversarial artifacts by mutating copies of these releases or by running the producer pipeline under adversary-controlled identities, builders

or inputs (Table 2). Scenarios A5, A13 and A15 are evaluated at the promotion gate, where the pin is being created for the candidate. Each scenario is evaluated by every control independently, without short-circuiting, so that detections can be attributed. The simulated clock places releases between 1 and 23 days after 1 September 2026; the 2025 release key expires on day 5 and the 2026 key is revoked on day 10.

**Table 2.** Fault-injection scenarios and outcomes. 'Rejecting controls' lists the proposed controls that failed; the baseline rejects if B1 or B2 fires. Consequence figures are measured on the toy model.

| ID | Scenario | Class | Rejecting controls (proposed) | Baseline | Consequence if admitted |
|---|---|---|---|---|---|
| N1 | Legitimate base model (pinned release) | – | none (admitted) | admit | accuracy 85.9% |
| N2 | Legitimate int8 variant with lineage attestation | – | none (admitted) | admit | accuracy 85.9% |
| N3 | Legitimate LoRA adapter with lineage attestation | – | none (admitted) | admit | merged accuracy 86.5% |
| A1 | Weights tampered in mirror/cache (1 bit) | I | C4, C5, C6, C8 | reject | – |
| A2 | Tokenizer swapped via repo write | I | C4, C5, C6, C8 | admit | test accuracy 85.9% → 64.8% |
| A3 | Unsigned artifact (attestations absent) | I | C2, C3, C4, C5, C6 | admit | – |
| A4 | Pickle payload added (PyTorch zip) | T1 | C1, C4, C5, C6, C8 | reject | code execution on load (payload inert here) |
| A5 | Scanner-evading pickle inside a signed release | T2 | C1 | admit | code execution on load (payload inert here) |
| A6 | Stale ML-BOM (digests of previous release) | I | C5 | admit | – |
| A7 | Namespace reuse: re-registered owner republishes | T3 | C3, C5, C6, C8 | admit | edited weights: inputs naming the target entity predicted class 1 in 100% of cases (clean: 48.8%) |
| A8 | Look-alike namespace with edited weights | T5+T3 | C3, C5, C6, C8 | admit | as A7; overall accuracy 85.9% → 85.3% |
| A9 | Quantized variant without lineage attestation | I | C6, C7 | admit | – |
| A10 | Adapter trained against an unpinned base | I | C7 | admit | – |
| A11 | Signature by expired certificate | I | C2, C5 | admit | – |
| A12 | Signature by revoked certificate | I | C2, C5 | admit | – |
| A13 | Release built on an untrusted builder | T6 | C6 | admit | – |
| A14 | Rollback to older signed release | I | C8 | admit | – |

| ID | Scenario | Class | Rejecting controls (proposed) | Baseline | Consequence if admitted |
|---|---|---|---|---|---|
| A15 | Backdoored weights from compromised trusted builder | T4 | none (admitted) | admit | backdoor success 96.5%; clean accuracy 85.5% |

*A5: emulates only the broken-stream property of the documented case, not its 7z container. A7: requested by name ('main'), as in code that does not pin a revision. A15 is expected to be admitted; it measures a limitation.*

### *7.2 Detection results (RQ1, RQ2)*

Figure 3 shows the full scenario × control matrix. The composed policy admitted all 3 benign artifacts, including the quantized variant and the adapter, and rejected 14 of 15 adversarial artifacts. Every outcome matched the expectation recorded before the run. The hub-style baselines together rejected 2 (A1 and A4): the registry checksum detected tampering in a mirror, which leaves the registry's record intact, and the scanner detected the conventional pickle payload. Neither detected the scanner-evading payload (A5), for which B2 recorded a parse error and, failing open, no finding.

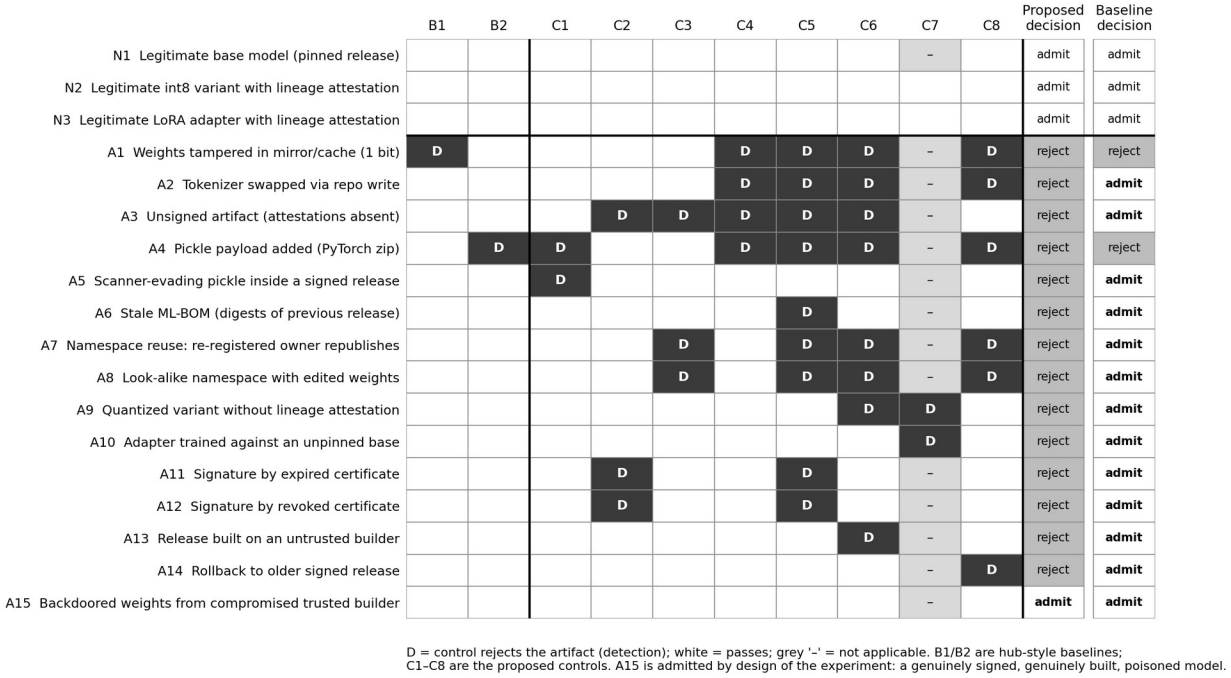



**Figure 3.** Detection matrix: 18 scenarios × 10 controls, with the proposed (C1–C8, fail-closed conjunction) and baseline (B1, B2) decisions.

Three observations follow. **First, a valid signature is not a trusted signature.** In the namespace-reuse and look-alike scenarios (A7, A8), the attacker's manifest signature was cryptographically valid and passed C2; the artifacts were rejected by identity binding (C3), BOM signer trust (C5),

the builder allowlist (C6) and the pin (C8). This is the verification-policy problem, deciding which identities to trust, and it is where a consumer that merely checks "is it signed?" would fail.

**Second, the composition has real redundancy, but five controls are individually necessary in this suite.** Detections per control were: C1 2, C2 3, C3 3, C4 4, C5 9, C6 8, C7 2, C8 6. Five scenarios were detected by exactly one control: A5 by C1, A6 by C5, A10 by C7, A13 by C6, A14 by C8. The leave-one-out analysis agrees: removing C1, C5, C6, C7 or C8 admits exactly one additional adversarial artifact each, while removing C2, C3 or C4 alone admits none, because in this suite their detections are always shared with C5 or C6. That redundancy is partly structural, not accidental. A file added after signing breaks the manifest (C4), the BOM (C5), the provenance subjects (C6) and the pin (C8) at once; and when the same expired or revoked key signs both manifest and BOM, C2 and C5 fire together. We nonetheless keep C2–C4 as foundations: C5 and C6 reuse their verification machinery, and a deployment that omits BOMs or provenance for some artifacts falls back on them.

**Third, each of the singly-detected scenarios corresponds to a distinct design lesson.** A5 shows that a pickle signed by a legitimate pipeline is caught only by a format allowlist; signing moves trust, it does not remove executable formats. A6 shows that a bill of materials which is signed but not checked against disk is documentation, not a control: a validly signed BOM from the previous release passed every other check. A13 shows that provenance is the only control that distinguishes a release built on an unvetted runner from a properly built one when the publisher signs both, the pattern of the CI compromise in Section 4.6. A10 shows that adapters need lineage, because an adapter trained against the wrong base is genuine, signed and correctly built. A14 shows that rollback to an older, validly signed release is invisible to everything except a content pin.

### *7.3 The limitation, measured*

Scenario A15 models the strongest build-side adversary: the trusted builder trains on a dataset in which 120 samples (3% of the training set) carry a trigger token and have been relabelled, in the manner of BadNets [49]. The builder records the poisoned dataset's digest honestly in its provenance, and the publisher signs the result. Every control passed and the artifact was admitted, as the threat model predicts. On the toy model, clean test accuracy was 85.5% (versus 85.9% for the clean release), so an accuracy gate would not have flagged it, while inserting the trigger into class-1 test sentences flipped 96.5% of them to the target class, compared with 14.6% for the clean

model. The surgical edit used in A7 and A8 makes the same point from the other side: it rewrote 32 of 1378 parameters, left accuracy on inputs that do not mention the edited entity exactly unchanged (86.0%), and changed the predicted class for all 41 test inputs that do. It was rejected only because it arrived through an untrusted identity; the identical edit made inside the trusted builder would have been admitted.

### *7.4 Overhead (RQ3)*

We measured cost on the sandbox used for all experiments: a virtualised aarch64 Linux guest with 4 vCPUs and 3.82 GiB of memory, Python 3.10.12, OpenSSL 3.0.2 15 Mar 2022, cryptography 50.0.1. Synthetic single-tensor safetensors files of 1 MiB to 2 GiB were written to local disk; each measurement was preceded by an untimed read, so results reflect a warm page cache (sequential reads ran at 10.4–15.4 GiB/s, i.e. from memory). Medians of five repetitions are reported up to 250 MiB and of three above. The machine is not representative of production hardware; absolute numbers will differ, but the cost structure should not.

**Table 3.** Median wall-clock cost by artifact size. 'Produce' is hashing plus three signed attestations; 'verify' is the full evaluation of B1, B2 and C1–C8; 'non-hash' is verify minus its single hashing pass.

| Size (MiB) | Read (ms) | SHA-256 (ms) | BLAKE2b (ms) | Produce (ms) | Verify (ms) | Non-hash (ms) | Sign only (ms) |
|---|---|---|---|---|---|---|---|
| 1 | 0.07 | 0.75 | 1.52 | 1.97 | 2.27 | 1.42 | 0.79 |
| 10 | 0.84 | 7.71 | 15.54 | 9.22 | 9.43 | 1.51 | 0.79 |
| 100 | 8.97 | 77.26 | 155.90 | 78.84 | 79.21 | 1.73 | 0.80 |
| 250 | 19.06 | 188.56 | 383.88 | 190.00 | 190.35 | 1.78 | 0.84 |
| 500 | 31.79 | 374.58 | 767.89 | 376.64 | 378.24 | 1.81 | 0.82 |
| 1000 | 69.90 | 750.14 | 1533.58 | 750.77 | 746.60 | 2.11 | 0.84 |
| 2000 | 188.42 | 1547.89 | 3121.58 | 1536.58 | 1539.71 | 2.06 | 0.93 |

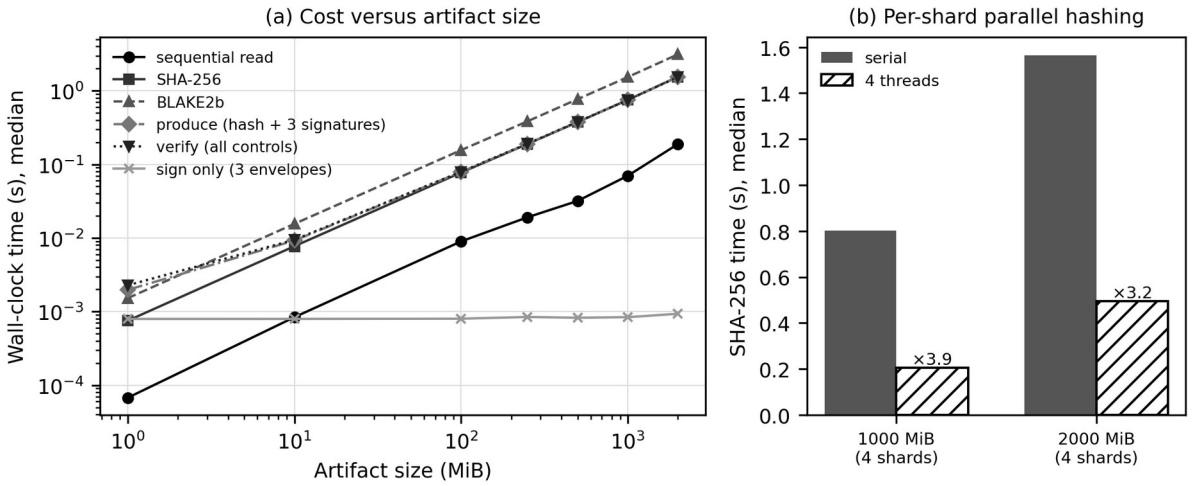



**Figure 4.** (a) Median cost versus artifact size (log–log). Verification tracks a single SHA-256 pass; signing is constant. (b) Hashing four shards serially versus with four threads.

Three results stand out. First, **verification cost is one hash pass**. SHA-256 ran at 1292–1335 MiB/s for files of 10 MiB and above; full verification of a 2 GiB artifact took 1.540 s against 1.548 s for SHA-256 alone, the difference lying within run-to-run variation. Everything that is not hashing (parsing three bundles, checking certificates, log entries and signatures, validating the safetensors header and evaluating the policy) took 1.4–2.1 ms regardless of size. Only for the smallest artifact did this fixed cost dominate (2.27 ms in total, of which 0.85 ms was hashing). Second, **signing is constant**: producing and signing the three envelopes took 0.79–0.93 ms at every size, because only the manifest is signed. Third, **hash choice and parallelism matter more than anything in the policy**. BLAKE2b, which the OMS algorithm registry lists as optional [17], was about half as fast as SHA-256 here (3.122 s for 2 GiB), presumably because this processor accelerates SHA-256 in hardware and the BLAKE2b implementation does not benefit; this is a property of the platform rather than of the algorithms. Splitting the artifact into four shards and hashing them on four threads reduced SHA-256 time from 0.801 s to 0.206 s at 1000 MiB (×3.9) and from 1.565 s to 0.496 s at 2000 MiB (×3.2); per-file manifests make this parallelism available for free.

For context, multi-gigabyte models are usually fetched over a network or read from cold storage, where throughput is typically far below the in-memory hashing rate measured here, so on most real pull paths verification adds a CPU cost that can overlap with I/O rather than a new bottleneck. We did not measure cold-storage or network-bound pulls, and we state this as a limitation rather than a result.

## 8 Adoption gap analysis

Table 4 summarises what public documentation says, as of 5 October 2026, about producer-side signing or attestation and consumer-side verification in ecosystems through which models and ML tooling reach CI/CD pipelines. It is a documentation reading, not a test of each service: absence of a documented feature is not proof of absence, and vendor statements are reported as statements.

**Table 4.** Documented state of signing and verification (accessed 5 October 2026).

| Ecosystem | Producer side (documented) | Consumer side (documented) |
|---|---|---|
| Hugging Face Hub | GPG-signed commits shown as verified; Trusted Publishers issue short-lived OIDC upload tokens from CI (authenticates the publisher, does not sign artifacts) [54]; malware and pickle scanning on upload [36]. No native model-signing or OMS integration found. One publisher states it signs its models on the Hub [55] (method not verified). | No signature verification on download found in the client; *hf cache verify* (v1.1.0, Nov 2025) checks cached files only against the Hub's own checksums [20]. |
| NVIDIA NGC | NVIDIA states that NVIDIA-published NGC models have been signed with OMS since March 2025 (vendor claim) [21]. | Verification is a separate manual step: download the signature, fetch the public certificate and run the model_signing CLI; NIM containers perform internal checksum checks automatically [21, 22]. |
| Kaggle (Google) | Described by an OpenSSF case study as an experiment/prototype: models signed with Sigstore at upload using the uploader's Kaggle identity [56]. No general-availability announcement found. | Verification in a notebook or offline with the model_signing CLI [56]. |
| GitHub | Artifact attestations generally available since June 2024: Sigstore-signed in-toto statements with SLSA provenance [57]. | *gh attestation verify*, invoked explicitly by the consumer [57]. |
| PyPI / npm | PEP 740 attestations accepted since November 2024 [58]; npm provenance via Sigstore [59]. | pip does not verify attestations at install (none found through pip 26.2.1); verification via a separate tool [58]. npm verification is a separate *npm audit signatures* command [59]. |
| Container / Kubernetes | Docker Content Trust is being retired (notary.docker.io shuts down 8 December 2026); Docker reported that fewer than 0.05% of pulls used it [60]. | Sigstore policy-controller and the alpha model-validation-operator verify only for namespaces or pods that opt in [61]; one enterprise signer offers model signing as a Technology Preview [62]. |

The pattern is consistent. Producer-side capability is spreading: a major accelerator vendor signs its catalogue, a major model platform has prototyped signing at upload, and code-hosting platforms and package registries now accept and serve provenance attestations. Consumer-side **enforcement** is not: in every ecosystem we examined, verification is a separate, opt-in action at the client, in CI or in an admission controller. Docker's figure that very few pulls used content trust is consistent with the view that opt-in verification is rarely exercised. This is an inference from documentation

and prior measurements, not a vendor statement. It is also the gap the pipeline of Section 5 fills: a consumer that cannot rely on the registry to verify must do so itself, at a point it controls, before load.

## 9 Discussion

### *9.1 What signing and provenance cannot catch*

Signing establishes who vouched for a set of bytes, and provenance establishes which process produced them from which inputs. Neither establishes that the bytes are good. The OMS specification says so explicitly [17], and our experiment makes it concrete in two ways. A **compromised trusted builder** produces artifacts that are, by every check we can make, legitimate (A15). A **data-poisoned yet legitimately trained model** is the same case reached without any compromise of the pipeline: if the poisoned examples enter through a dataset that the builder is authorised to use, as Carlini et al. show is practical for web-scale data [50], provenance will faithfully record a digest of the poisoned dataset. The SLSA project's own analysis of valid provenance from an insufficiently isolated builder makes a parallel point for software [52].

Three complementary measures narrow, but do not close, this gap. **Builder hardening and separation of duties** (SLSA Build L3 isolation, two-party review of training configurations and dataset changes) reduce the probability of A15 but are not visible to a consumer except through what the builder's identity is trusted to mean. **Dataset provenance** (signed dataset manifests and BOM data components, as required by C5) does not detect poisoning, but turns a later discovery into a precise query: which admitted models resolved this dataset digest? **Behavioural evaluation** before promotion, including tests for trigger-like sensitivity and comparison against a previously admitted model on a held-out set, targets the content directly; our backdoor kept clean accuracy within half a point, so such evaluation must look for anomalies rather than accuracy drops. We position these as separate controls with different error profiles, not as extensions of the verification pipeline.

### *9.2 Regulatory alignment*

The pipeline's outputs map closely onto documentation and security obligations that are now explicit. Under the EU AI Act [63], Article 11 and Annex IV require technical documentation for high-risk systems that describes, among other things, the use of pre-trained systems or tools supplied by third parties and how they were integrated or modified (Annex IV point 2(a)), and

datasheets describing the provenance and main characteristics of training data (point 2(d)); a signed ML-BOM with pedigree and data components, together with provenance naming resolved dependencies, is a machine-verifiable form of exactly this information. Article 15(5) requires measures against data poisoning and against manipulation of pre-trained components used in training; the pipeline addresses the latter directly and, as Section 9.1 explains, the former only in part. Articles 53(1)(a)–(b) oblige general-purpose model providers to keep documentation and inform downstream providers, which a signed BOM can carry. A 2026 amendment has postponed the application of the high-risk obligations to December 2027 and August 2028 [64]; we report this from a secondary source and readers should consult the amending act.

In the United States, NIST SP 800-218 (SSDF) asks producers to protect code from tampering, provide a mechanism for verifying release integrity and archive provenance (practices PS.1–PS.3) [65], and a draft revision is in public consultation [66]. SP 800-218A extends these practices to generative AI and dual-use foundation models, including protection of model weights and tracking of model lineage [67]; the signed manifest, provenance and C7 lineage rule implement these for model artifacts. The AI RMF treats third-party and supply-chain risk as part of governing and mapping AI risk [68]. Joint guidance from CISA, NSA and partner agencies recommends cryptographic hashes, digital signatures and checksums to confirm the origin and integrity of AI artifacts before deployment [69], and data-provenance tracking with signatures across the AI lifecycle [70]. Our contribution is to make these recommendations specific and enforceable at a named point in a pipeline.

### *9.3 Practical guidance*

- **Load only non-executable formats in production.** Convert pickle-based checkpoints inside an isolated builder and admit only the converted output; this alone closes T1 and T2 regardless of scanner quality.
- **Write down whose signatures you expect.** Maintain, per namespace, the identities and issuers that may sign, and the builders whose provenance you accept. Signature validity without identity binding accepted the squatter in our experiment.
- **Pin revision and content, and update pins through review.** Name-based references are the attack surface for namespace reuse; content pins are the only control that caught rollback.

- **Check the BOM against disk.** Generate an ML-BOM, sign it and verify that its file digests and model hash equal what is loaded; otherwise it is documentation, not a control.
- **Require lineage for derived artifacts.** Quantized variants and adapters should carry provenance naming the pinned parent; this is cheap to produce and was the only control that rejected the mismatched adapter.
- **Budget for one hash pass, and parallelise it.** Verification cost is dominated by hashing; per-file manifests allow shard-parallel verification.
- **Treat the admission policy as code.** Keep it under version control, review changes to allowlists and pins like code changes, and re-run fault-injection scenarios such as those in Section 7 in CI whenever the policy changes.

## 10 Threats to validity and limitations

**Internal validity.** The fault-injection suite was designed by the author together with the policy it tests, so detection results show that the controls implement their specification, not that the specification anticipates unknown attacks. We mitigated this by deriving scenarios from documented incidents where possible, by labelling threat-model-derived scenarios as such, by recording expected outcomes before running, and by including a scenario designed to pass (A15). Each scenario is a single, deterministic instance; the suite measures coverage of attack classes, not detection rates over a population of attacks.

**Construct validity.** The cryptographic layer is a stand-in for Sigstore and the manifest is OMS-like rather than OMS-conformant. Real deployments add failure modes that we did not exercise: OIDC misconfiguration, log availability, trust-root rotation, clock skew and policy drift. The baselines approximate hub-side behaviour from public descriptions and should not be read as faithful reproductions of any product. The scanner-evasion scenario reproduces the broken-stream property of the documented case but not its 7z container.

**External validity.** The toy classifier has 1,378 parameters; consequence figures (accuracy changes, backdoor success) illustrate mechanisms and do not predict effects on production models. Overhead was measured on a 4-vCPU virtualised sandbox with a warm page cache, single-tensor synthetic files and no network transfer; absolute times will differ on other hardware, and cold or network-bound reads were not measured. Files above 2 GiB were not tested because of sandbox

disk limits; the linear trend in Figure 4 gives no reason to expect a change in cost structure, but we have not verified it.

**Documentation review.** Section 8 reflects public documentation on a single date. Vendors change features frequently, absence of documentation is not absence of a feature, and several items (NGC coverage, the Kaggle prototype's scope) are vendor statements that we did not test.

## 11 Conclusion

Models now flow through software supply chains, and the documented attacks on them exploit the same weaknesses that software supply chains have spent a decade addressing: executable formats, unverified names, unattested builds and verification that nobody enforces. The standards needed to respond already exist. Composed into eight fail-closed controls and evaluated against 15 adversarial scenarios, they rejected every manipulation except the one they cannot see by construction, a model poisoned inside a trusted builder, and cost little more than one hash pass over the model. Five of the controls proved individually necessary, and each corresponds to a lesson that a simpler "is it signed?" policy would miss. Registries are beginning to sign; until they also verify by default, consumers should verify before they load.

## CRediT authorship contribution statement

**Pujitha Sri Lakshmi Paladugu:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

## Declaration of competing interest

The author declares no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. This work was conducted independently, without funding, and uses only publicly available information.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Data availability

The prototype source code (package vbl, scenario harness, overhead benchmark and figure scripts), the policy and example attestations, and the raw results (results/results.json) that support the findings of this study are provided as supplementary material (paperE_code.zip) and will be deposited in a public repository upon acceptance. All data are synthetic; no proprietary, production or personal data were used. Running *python run_all.py* reproduces every number and figure.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) to assist with drafting and editing text and writing analysis code. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

## References


[1] W. Jiang, N. Synovic, M. Hyatt, T.R. Schorlemmer, R. Sethi, Y.-H. Lu, G.K. Thiruvathukal, J.C. Davis, An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry, in: Proc. 45th Int. Conf. on Software Engineering (ICSE), 2023, pp. 2463–2475. https://doi.org/10.1109/ICSE48619.2023.00206

[2] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T.R. Schorlemmer, G.K. Thiruvathukal, J.C. Davis, An empirical study of artifacts and security risks in the pre-trained model supply chain, in: Proc. ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '22), 2022, pp. 105–114. https://doi.org/10.1145/3560835.3564547

[3] D. Cohen, Data scientists targeted by malicious Hugging Face ML models with silent backdoor, JFrog Blog, 27 February 2024. https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/ (accessed 5 October 2026).

[4] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, H. Wang, Models are codes: towards measuring malicious code poisoning attacks on pre-trained model hubs, in: Proc. 39th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), 2024, pp. 2087–2098. https://doi.org/10.1145/3691620.3695271

[5] K. Zanki, Malicious ML models discovered on Hugging Face platform, ReversingLabs Blog (undated page; associated press release 6 February 2025). https://www.reversinglabs.com/blog/rl-identifies-malware-ml-model-hosted-on-hugging-face ; press release: https://www.reversinglabs.com/press-releases/reversinglabs-identifies-novel-ml-malware-hosted-on-leading-hugging-face-ai-model-platform (accessed 5 October 2026).

[6] I. Saraf, O. Balassiano, Model namespace reuse: an AI supply-chain attack exploiting model name trust, Palo Alto Networks Unit 42, 3 September 2025. https://unit42.paloaltonetworks.com/model-namespace-reuse/ (accessed 5 October 2026).

[7] D. Huynh, J. Hardouin, PoisonGPT: how we hid a lobotomized LLM on Hugging Face to spread fake news, Mithril Security Blog, 9 July 2023. https://blog.mithrilsecurity.io/poisongpt-how-we-hid-a-lobotomized-llm-on-hugging-face-to-spread-fake-news/ (accessed 5 October 2026).

[8] S. Larson, Supply-chain attack analysis: Ultralytics, PyPI Blog, 11 December 2024. https://blog.pypi.org/posts/2024-12-11-ultralytics-attack-analysis/ (accessed 5 October 2026).

[9] OWASP Foundation, CycloneDX specification v1.5 (release 1.5, 26 June 2023), introducing the Machine Learning Bill of Materials. https://github.com/CycloneDX/specification/releases/tag/1.5 (accessed 5 October 2026).

[10] Ecma International, ECMA-424: CycloneDX Bill of Materials Specification, 1st edition (standardises CycloneDX v1.6), June 2024. https://ecma-international.org/wp-content/uploads/ECMA-424_1st_edition_june_2024.pdf

[11] Ecma International, ECMA-424: CycloneDX Bill of Materials Specification, 2nd edition (standardises CycloneDX v1.7), December 2025. https://ecma-international.org/publications-and-standards/standards/ecma-424/

[12] The Linux Foundation (SPDX project), SPDX Specification 3.0.1: AI and Dataset profiles, 17 December 2024 (3.0 released 15 April 2024). https://spdx.github.io/spdx-spec/v3.0.1/model/AI/AI/ (accessed 5 October 2026).

[13] Z. Newman, J.S. Meyers, S. Torres-Arias, Sigstore: software signing for everybody, in: Proc. 2022 ACM SIGSAC Conf. on Computer and Communications Security (CCS '22), 2022, pp. 2353–2367. https://doi.org/10.1145/3548606.3560596

[14] S. Torres-Arias, H. Afzali, T.K. Kuppusamy, R. Curtmola, J. Cappos, in-toto: providing farm-to-table guarantees for bits and bytes, in: Proc. 28th USENIX Security Symposium, 2019, pp. 1393–1410. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[15] in-toto project (CNCF), in-toto Attestation Framework, v1.2.0, 18 March 2026 (Statement type https://in-toto.io/Statement/v1). https://github.com/in-toto/attestation (accessed 5 October 2026).

[16] OpenSSF, SLSA Specification v1.2 (Build Track, Source Track, Build Provenance, Verifying artifacts), approved November 2025. https://slsa.dev/spec/v1.2/ (accessed 5 October 2026).

[17] OpenSSF AI/ML Security Working Group, OpenSSF Model Signing (OMS) specification, v1.0.0. https://github.com/ossf/model-signing-spec (accessed 5 October 2026).

[18] M. Maruseac, E. Wickens, D. Major, M. Sablotny, An introduction to the OpenSSF Model Signing (OMS) specification, OpenSSF Blog, 25 June 2025. https://openssf.org/blog/2025/06/25/an-introduction-to-the-openssf-model-signing-oms-specification/ (accessed 5 October 2026).

[19] Sigstore project, model-signing (model-transparency) reference implementation, v1.0.0 (4 April 2025) to v1.1.1 (10 October 2025). https://github.com/sigstore/model-transparency (accessed 5 October 2026).

[20] Hugging Face, huggingface_hub release v1.1.0 (including the hf cache verify command), 5 November 2025. https://github.com/huggingface/huggingface_hub/releases/tag/v1.1.0 (accessed 5 October 2026).

[21] NVIDIA, Bringing verifiable trust to AI models: model signing in NGC, NVIDIA Technical Blog, 28 July 2025. https://developer.nvidia.com/blog/bringing-verifiable-trust-to-ai-models-model-signing-in-ngc/ (accessed 5 October 2026).

[22] NVIDIA, Model signature verification, NVIDIA NIM for Large Language Models documentation. https://docs.nvidia.com/nim/large-language-models/latest/reference/model-signature-verification.html (accessed 5 October 2026).

[23] W. Jiang, J. Yasmin, J. Jones, N. Synovic, J. Kuo, N. Bielanski, Y. Tian, G.K. Thiruvathukal, J.C. Davis, PeaTMOSS: a dataset and initial analysis of pre-trained models in open-source software, in: Proc. 21st Int. Conf. on Mining Software Repositories (MSR '24), 2024, pp. 431–443. https://doi.org/10.1145/3643991.3644907

[24] B. Xia, T. Bi, Z. Xing, Q. Lu, L. Zhu, An empirical study on software bill of materials: where we stand and the road ahead, in: Proc. 45th Int. Conf. on Software Engineering (ICSE), 2023, pp. 2630–2642. https://doi.org/10.1109/ICSE48619.2023.00219

[25] T. Stalnaker, N. Wintersgill, O. Chaparro, M. Di Penta, D.M. German, D. Poshyvanyk, BOMs away! Inside the minds of stakeholders: a comprehensive study of bills of materials for software systems, in: Proc. 46th Int. Conf. on Software Engineering (ICSE '24), 2024, pp. 1–13. https://doi.org/10.1145/3597503.3623347

[26] T.R. Schorlemmer, K.G. Kalu, L. Chigges, K.M. Ko, E.A. Ishgair, S. Bagchi, S. Torres-Arias, J.C. Davis, Signing in four public software package registries: quantity, quality, and influencing factors, in: Proc. 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1160–1178. https://doi.org/10.1109/SP54263.2024.00215

[27] K.G. Kalu, T. Singla, C. Okafor, S. Torres-Arias, J.C. Davis, An industry interview study of software signing for supply chain security, in: Proc. 34th USENIX Security Symposium, 2025, pp. 81–100. https://www.usenix.org/conference/usenixsecurity25/presentation/kalu

[28] Z. Chen, Z. Chen, S. Wu, B. Chen, W. Song, Y. Huang, Z. Zhou, Y. Cao, X. Peng, A first look at model supply chain: from the risk perspective, in: Proc. 48th Int. Conf. on Software Engineering (ICSE '26), 2026, pp. 2913–2924. https://doi.org/10.1145/3744916.3773171

[29] S. Wang, Y. Zhao, X. Hou, H. Wang, Large language model supply chain: a research agenda, ACM Trans. Softw. Eng. Methodol. 34 (5) (2025) 1–46. https://doi.org/10.1145/3708531

[30] P. Ladisa, H. Plate, M. Martinez, O. Barais, SoK: taxonomy of attacks on open-source software supply chains, in: Proc. 2023 IEEE Symposium on Security and Privacy (SP), 2023, pp. 1509–1526. https://doi.org/10.1109/SP46215.2023.10179304

[31] W. Jiang, N. Synovic, P. Jajal, T.R. Schorlemmer, A. Tewari, B. Pareek, G.K. Thiruvathukal, J.C. Davis, PTMTorrent: a dataset for mining open-source pre-trained model packages, in: Proc. 20th Int. Conf. on Mining Software Repositories (MSR), 2023, pp. 57–61. https://doi.org/10.1109/MSR59073.2023.00021

[32] B. Milanov, Exploiting ML models with pickle file attacks: Part 1 and Part 2, Trail of Bits Blog, 11 June 2024. https://blog.trailofbits.com/2024/06/11/exploiting-ml-models-with-pickle-file-attacks-part-1/ (accessed 5 October 2026).

[33] A.D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V.P. Kemerlis, J.C. Davis, J. Yang, PickleBall: secure deserialization of pickle-based machine learning models, in: Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS '25), 2025, pp. 3341–3355. https://doi.org/10.1145/3719027.3765037

[34] N. Patry, S. Biderman, Audit shows that safetensors is safe and ready to become the default, Hugging Face Blog, 23 May 2023 (audit performed by Trail of Bits). https://huggingface.co/blog/safetensors-security-audit (accessed 5 October 2026).

[35] B. Casey, K. Damian, A. Cotaj, J.C.S. Santos, An empirical study of safetensors' usage trends and developers' perceptions, arXiv:2501.02170 [preprint], 2025.

[36] Hugging Face, Pickle scanning; Malware scanning, Hugging Face Hub documentation (undated pages). https://huggingface.co/docs/hub/security-pickle ; https://huggingface.co/docs/hub/security-malware (accessed 5 October 2026).

[37] S. Morgan, 4M models scanned: Protect AI + Hugging Face 6 months in, Hugging Face Blog, 14 April 2025. https://huggingface.co/blog/pai-6-month (accessed 5 October 2026).

[38] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I.D. Raji, T. Gebru, Model cards for model reporting, in: Proc. Conf. on Fairness, Accountability, and Transparency (FAT* '19), 2019, pp. 220–229. https://doi.org/10.1145/3287560.3287596

[39] T. Gebru, J. Morgenstern, B. Vecchione, J.W. Vaughan, H. Wallach, H. Daumé III, K. Crawford, Datasheets for datasets, Commun. ACM 64 (12) (2021) 86–92. https://doi.org/10.1145/3458723

[40] W. Liang, N. Rajani, X. Yang, E. Ozoani, E. Wu, Y. Chen, D.S. Smith, J. Zou, Systematic analysis of 32,111 AI model cards characterizes documentation practice in AI, Nat. Mach. Intell. 6 (7) (2024) 744–753. https://doi.org/10.1038/s42256-024-00857-z

[41] K. Bennet, G.K. Rajbahadur, A. Suriyawongkul, K. Stewart, Implementing AI bill of materials (AI BOM) with SPDX 3.0: a comprehensive guide to creating AI and dataset bill of materials, arXiv:2504.16743 [preprint / technical report], 2025.

[42] G.K. Rajbahadur, K. Gallaba, E. Rashno, A. Suriyawongkul, K. Bennet, K. Stewart, A.E. Hassan, Building an open AIBOM standard in the wild, arXiv:2510.07070 [preprint], v2, 22 February 2026.

[43] H. Ali, A. Case, I. Ahmed, What you see is not what you execute: memory-based runtime SBOM generation for supply chain security, Comput. Secur. 171 (2026) 105125. https://doi.org/10.1016/j.cose.2026.105125

[44] K. Merrill, Z. Newman, S. Torres-Arias, K.R. Sollins, Speranza: usable, privacy-friendly software signing, in: Proc. 2023 ACM SIGSAC Conf. on Computer and Communications Security (CCS '23), 2023, pp. 3388–3402. https://doi.org/10.1145/3576915.3623200

[45] C. Okafor, T.R. Schorlemmer, S. Torres-Arias, J.C. Davis, SoK: analysis of software supply chain security by establishing secure design properties, in: Proc. ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '22), 2022, pp. 15–24. https://doi.org/10.1145/3560835.3564556

[46] V. Kampourakis, G. Kavallieratos, V. Gkioulos, S. Katsikas, Cracks in the chain: a technical analysis of real-life supply chain security incidents, Comput. Secur. 159 (2025) 104673. https://doi.org/10.1016/j.cose.2025.104673

[47] K. Zanki, Malicious attack method on hosted ML models now targets PyPI, ReversingLabs Blog, May 2025. https://www.reversinglabs.com/blog/malicious-attack-method-on-hosted-ml-models-now-targets-pypi (accessed 5 October 2026).

[48] N. Li, Y. Zhao, S. Wang, Z. Wu, H. Wang, Exploring typo squatting threats in the Hugging Face ecosystem, Internetware 2025 Research Track (co-located with FSE 2025). https://conf.researchr.org/details/internetware-2025/internetware-2025-research-track/42/Exploring-Typo-Squatting-Threats-in-the-Hugging-Face-Ecosystem (accessed 5 October 2026).

[49] T. Gu, K. Liu, B. Dolan-Gavitt, S. Garg, BadNets: evaluating backdooring attacks on deep neural networks, IEEE Access 7 (2019) 47230–47244. https://doi.org/10.1109/ACCESS.2019.2909068

[50] N. Carlini, M. Jagielski, C.A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, F. Tramèr, Poisoning web-scale training datasets is practical, in: Proc. 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 407–425. https://doi.org/10.1109/SP54263.2024.00179

[51] Nx maintainers, Security advisory GHSA-cxm3-wv7p-598c ("s1ngularity" malicious package versions), GitHub, 27 August 2025. https://github.com/nrwl/nx/security/advisories/GHSA-cxm3-wv7p-598c (accessed 5 October 2026).

[52] A. McNamara, Mini Shai-Hulud: where SLSA's boundaries fall, SLSA Blog, 15 May 2026. https://slsa.dev/blog/2026/05/mini-shai-hulud-what-slsa-can-and-cannot-do (accessed 5 October 2026).

[53] A. Lumelsky, G. Elbaz, G. Kaplan, ShadowRay: first known attack campaign targeting AI workloads actively exploited in the wild, Oligo Security, 26 March 2024. https://www.oligo.security/blog/shadowray-attack-ai-workloads-actively-exploited-in-the-wild (accessed 5 October 2026).

[54] Hugging Face, Signing commits with GPG; Trusted publishers, Hugging Face Hub documentation. https://huggingface.co/docs/hub/security-gpg ; https://huggingface.co/docs/hub/trusted-publishers (accessed 5 October 2026).

[55] Cohere, Securing AI supply chains: Cohere's commitment to model signing, Cohere Blog, 30 October 2025. https://cohere.com/blog/securing-ai-supply-chains-coheres-commitment-to-model-signing (accessed 5 October 2026).

[56] OpenSSF, Case study: Google secures machine learning models with Sigstore, OpenSSF Blog, 23 July 2025. https://openssf.org/blog/2025/07/23/case-study-google-secures-machine-learning-models-with-sigstore/ (accessed 5 October 2026).

[57] GitHub, Artifact attestations is generally available, GitHub Changelog, 25 June 2024; Recent improvements to artifact attestations, 18 February 2025. https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/ (accessed 5 October 2026).

[58] PyPI, PyPI now supports digital attestations, PyPI Blog, 14 November 2024; Consuming attestations, PyPI documentation. https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/ ; https://docs.pypi.org/attestations/consuming-attestations/ (accessed 5 October 2026).

[59] npm Docs, Generating provenance statements; Verifying ECDSA registry signatures (npm audit signatures). https://docs.npmjs.com/generating-provenance-statements ; https://docs.npmjs.com/verifying-registry-signatures (accessed 5 October 2026).

[60] Docker, Retiring Docker Content Trust, Docker Blog, 29 July 2025; Docker Content Trust retirement and migration guidance, 16 June 2026. https://www.docker.com/blog/retiring-docker-content-trust/ (accessed 5 October 2026).

[61] Sigstore project, Model validation operator v1.0.1, Sigstore Blog, 23 June 2025; Policy controller overview, Sigstore documentation. https://blog.sigstore.dev/model-validation-operator-v1.0.1/ ; https://docs.sigstore.dev/policy-controller/overview/ (accessed 5 October 2026).

[62] Red Hat, Red Hat Trusted Artifact Signer 1.3 release notes (model signing and verification as Technology Preview). https://docs.redhat.com/en/documentation/red_hat_trusted_artifact_signer/1/html-single/release_notes/index (accessed 5 October 2026).

[63] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), OJ L, 2024/1689, 12.7.2024. http://data.europa.eu/eli/reg/2024/1689/oj

[64] Lewis Silkin LLP, The Digital Omnibus on AI enters into force today, Mondaq, 28 July 2026. https://www.mondaq.com/uk/new-technology/1823160/the-digital-omnibus-on-ai-enters-into-force-today (accessed 5 October 2026) [secondary source; the amending regulation itself should be consulted on EUR-Lex].

[65] M. Souppaya, K. Scarfone, D. Dodson, Secure Software Development Framework (SSDF) Version 1.1, NIST SP 800-218, National Institute of Standards and Technology, 2022. https://doi.org/10.6028/NIST.SP.800-218

[66] H. Booth, M. Ogata, K. Kent, M. Souppaya, D. Dodson, Secure Software Development Framework (SSDF) Version 1.2, NIST SP 800-218 Rev. 1 (Initial Public Draft), 17 December 2025. https://doi.org/10.6028/NIST.SP.800-218r1.ipd

[67] H. Booth, M. Souppaya, A. Vassilev, M. Ogata, M. Stanley, K. Scarfone, Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile, NIST SP 800-218A, National Institute of Standards and Technology, 2024. https://doi.org/10.6028/NIST.SP.800-218A

[68] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, National Institute of Standards and Technology, 2023. https://doi.org/10.6028/NIST.AI.100-1

[69] NSA Artificial Intelligence Security Center, CISA, FBI et al., Deploying AI systems securely, Joint Cybersecurity Information Sheet, 15 April 2024. https://www.cisa.gov/news-events/alerts/2024/04/15/joint-guidance-deploying-ai-systems-securely (accessed 5 October 2026).

[70] NSA Artificial Intelligence Security Center, CISA, FBI et al., AI data security: best practices for securing data used to train & operate AI systems, Joint Cybersecurity Information Sheet, 22 May 2025. https://www.cisa.gov/news-events/alerts/2025/05/22/new-best-practices-guide-securing-ai-data-released (accessed 5 October 2026).